Idea 2, step 3: every search step works on the rule

each round the search must …	with a list (Idea 1)	with a rule (Idea 2)
find the next states	follow every move of every state	one calculation (next slide)
add them to what we have	append them one by one	OR the two rules
check: a bad state among them?	look at every state	AND with the bad rule: FALSE?
check: nothing new?	compare the lists	is the rule the same?
work grows with …	**the number of states**	**the length of the rule**

Round 3, the check in one line: `¬(C1 ∧ C2) ∧ (C1 ∧ C2) = FALSE`
no bad state, without looking at the 8 states one by one.

> **The work follows the length of the rules, not the number of states: that is how Idea 2 fights the explosion.**

Idea 2, step 4: but every state has several moves!

Objection: each state has several moves → don't we still follow every move of every state?

The teacher does not move students

group: rows 1–5
move: each student stays or goes one row back
result: rows 1–6, calculated on the rule

2 numbers changed, not 150 students moved

reached after round 2: (¬C1∧¬C2) ∨ (C1∧N2) ∨ (N1∧C2)

+ T = all moves of both robots, one formula

one calculation on the two formulas

reached after round 3: ¬(C1 ∧ C2)

No (state, move) pair is ever listed. The cost grows with the size of the formulas, not with states × moves.

How the calculation works step by step: part ②, "AND, try both, rename".

All moves are one formula; all next states come out of one calculation.

Idea 2, step 5: how much work in total?

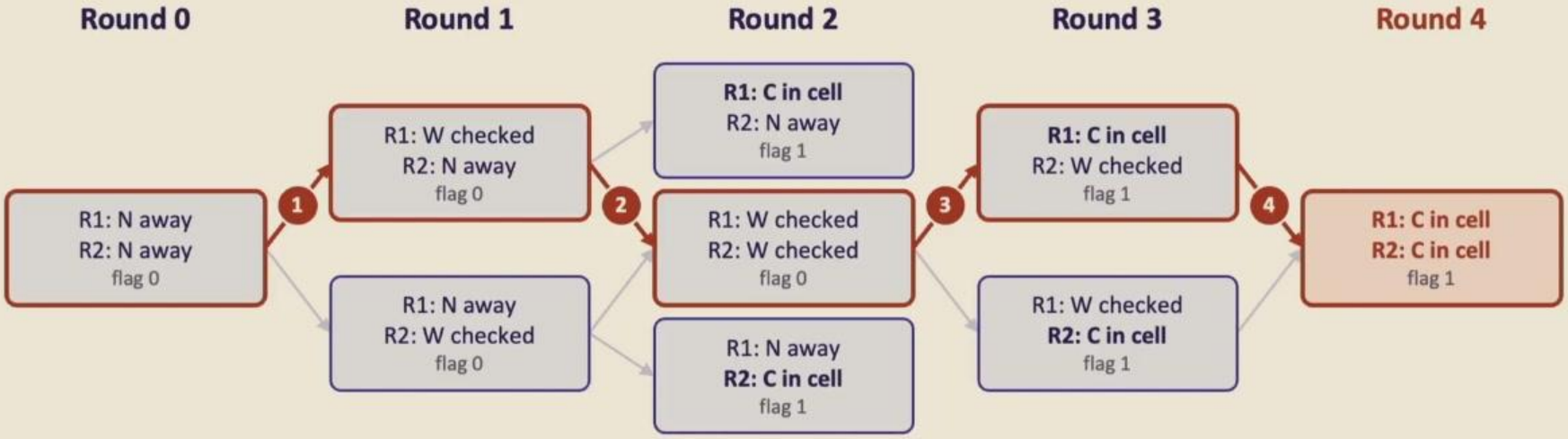
	Idea 1: one state at a time	Idea 2: one group at a time
one unit of work	tick one state	one round: next states, OR, AND, compare
how many units	number of reachable states	number of rounds = steps to the farthest state
cost of one unit	tiny	grows with the length of the formulas

example	states	Idea 1: ticks	Idea 2: rounds
two robots	9	9	5 until bug (rounds 0–4)
30 switches, set freely each scan	≈ 1 billion	≈ 1 billion	**3 (all reached in round 1)**
30-bit counter, +1 per scan	≈ 1 billion	≈ 1 billion	**≈ 1 billion: no gain**

Rounds, not paths: there may be millions of runs, but the search only goes as deep as the farthest state.

Idea 2 wins when a system has many states but few rounds, and the formulas stay short.

Idea 1: spread out step by step, like ripples in water



box = state of both robots · **arrow** = one robot moves · **red:** ① R1 checks ② R2 checks ③ R1 enters ④ R2 enters

Ask Before each round: which states come next?

Each round reaches everything one step further, until a bad state appears or nothing new does.

Idea 2, step 6: three questions to answer

① Describe

How can a formula describe a group of states, and when is it short?

② Step

How do we take one step with the whole group, without listing it?

③ Store

How does the computer store formulas and compare them fast?
→ BDDs

Like a teacher: "rows 1–5, please stand up" instead of calling 150 names one by one.

> **Symbolic model checking = the same ripple search, but each round is one formula instead of a list of states.**

① A rule describes a group, not a list

student	born in
Li Wei	**Oct**
Wang Fang	Mar
Chen Jie	**Oct**
Liu Yang	Jun
Zhao Lei	**Oct**
Sun Hui	Dec

Who was born in October?

List them

Li Wei, Chen Jie,
Zhao Lei

grows with the group

Give a rule

"born in October"

same length for 3 or 3,000 students

For PLC states: the rule is a logic formula, e.g. `full` = "all states where the tank is full".

Ask The class has 3,000 students. Which answer would you rather write down?

A rule describes a whole group, and stays short however big the group is.

① A formula picks rows of the truth table

row	full	filling	estop
1	0	0	0
2	0	0	1
3	**0**	**1**	**0**
4	0	1	1
5	**1**	**0**	**0**
6	**1**	**0**	**1**
7	**1**	**1**	**0**
8	**1**	**1**	**1**

`full`

picks the 4 blue rows: tank full

`filling ∧ ¬estop ∧ ¬full`

picks the 1 gold row: valve open

3 on/off signals → 2 × 2 × 2 = 8 rows = 8 states.

Ask Which rows does ¬full ∧ filling pick?

A formula is a rule that picks rows: all the states where it is TRUE.

① The same groups on a Karnaugh map

8 states as a Karnaugh map (each cell = one row of the table)

full \ filling estop	00	01	11	10
0				valve
1	full	full	full	full

blue rectangle: `full` (4 states)

gold cell: `filling ∧ ¬estop ∧ ¬full` (1 state)

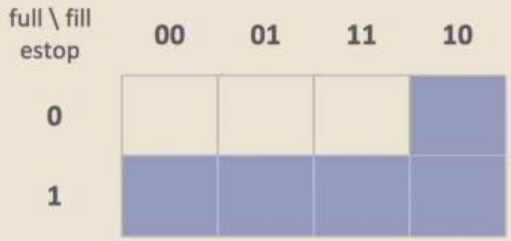
As in circuit design: one rectangle = one short product term.

How to read the map: the row gives full (0 / 1); the column gives filling and estop (order 00, 01, 11, 10, as in digital logic).

On a K-map, a formula is a rectangle of states. Bigger rectangles often mean shorter formulas.

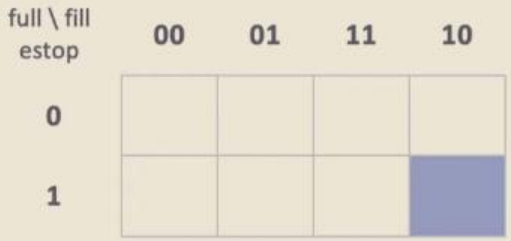
① Combine groups with OR, AND, NOT

A ∨ B (union)



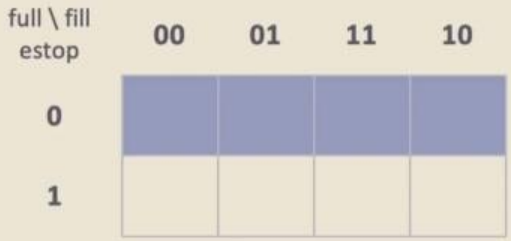
`full ∨ (filling ∧ ¬estop)`
`5 states`

A ∧ B (overlap)



`full ∧ filling ∧ ¬estop`
`1 state`

¬A (the rest)



`¬full`
`4 states`

A = `full` (bottom row) **B =** `filling ∧ ¬estop` (column 10)

Union = OR, overlap = AND, the rest = NOT. A model checker computes with groups using only logic.

① Why it pays: a short formula for a huge group

A panel with 30 on/off switches s1 … s30: about 1 billion states

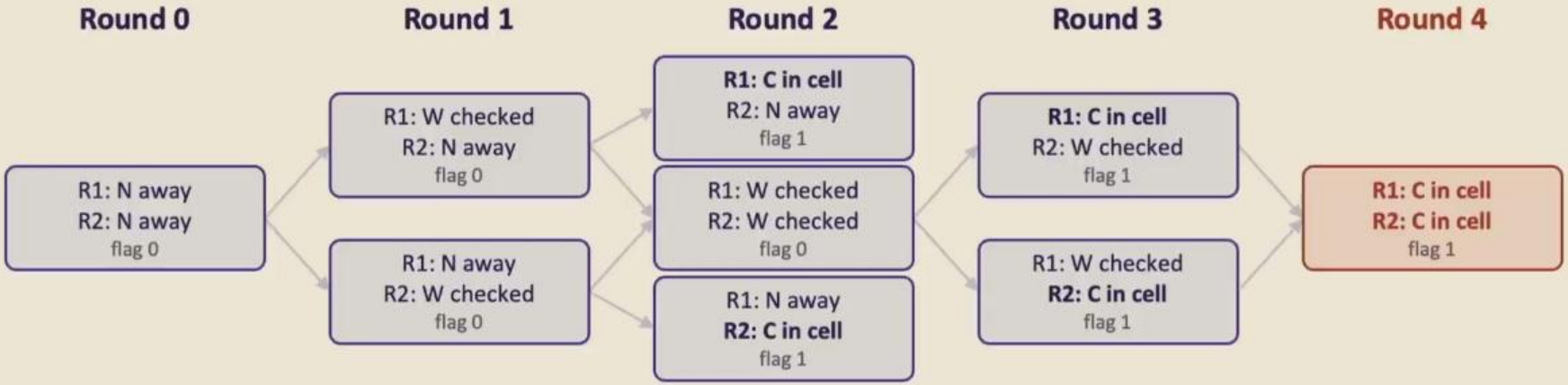
group of states	how many states	formula
all states	$2^{30} \approx 1$ billion	`TRUE`
switch 1 off	$2^{29} \approx 537$ million	`¬s1`
switches 1 and 2 off	$2^{28} \approx 268$ million	`¬s1 ∧ ¬s2`
all switches off	1	`¬s1 ∧ ¬s2 ∧ … ∧ ¬s30`

Not always: some groups need long formulas; symbolic tools win when the system has structure.

Ask The robots have 9 states (the 9 boxes of the ripple picture). How many does ¬(C1 ∧ C2) cover?

The work depends on how long the formula is, not on how many states it covers.

② Ripples again, but each round is one formula



boxes: the states that are new in each round · below: everything reached so far, as one formula

Round 0	Round 1	Round 2	Round 3	Round 4
`N1 ∧ N2`	`¬C1 ∧ ¬C2 ∧ ¬(W1 ∧ W2)`	`(¬C1 ∧ ¬C2) ∨ (C1 ∧ N2) ∨ (N1 ∧ C2)`	`¬(C1 ∧ C2)`	`TRUE = all 9, with C1 ∧ C2` ✗

Fixed protocol: after round 1 nothing new is added (a "fixed point"), and C1 ∧ C2 was never reached: holds. What happens between the rounds: next slide.

Same rounds, but the tool keeps one formula for everything reached, however many states it covers.


USB-C
IN : NO SIGNAL
OUT : 3840X2160P 60


No Signal



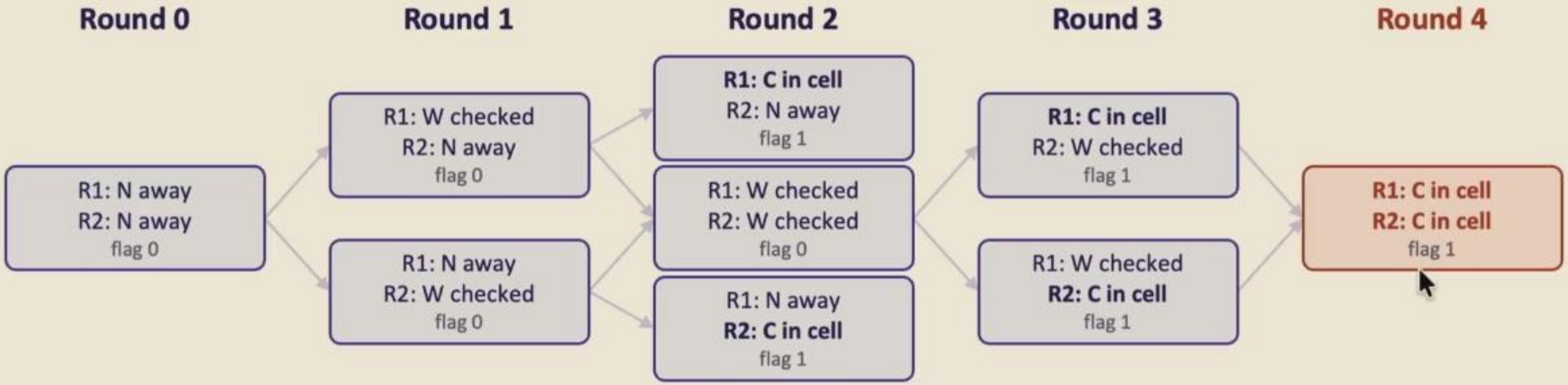
24.9cm X 25.4cm
冊頁設色
台北故宮博物院藏

《苹婆山鸟图》
黄筌 · 宋代

14:11
2026/10/26

USB-C
IN : 1920X1080P 60
OUT : 3840X2160P 60

(2) Ripples again, but each round is one formula



boxes: the states that are new in each round · below: everything reached so far, as one formula

Round 0	Round 1	Round 2	Round 3	Round 4
$N1 \wedge N2$	$\neg C1 \wedge \neg C2 \wedge \neg(W1 \wedge W2)$	$(\neg C1 \wedge \neg C2) \vee (C1 \wedge N2) \vee (N1 \wedge C2)$	$\neg(C1 \wedge C2)$	TRUE = all 9, with $C1 \wedge C2$ ✗

Fixed protocol: after round 1 nothing new is added (a "fixed point"), and C1 ∧ C2 was never reached: holds. What happens between the rounds: next slide.

Same rounds, but the tool keeps one formula for everything reached, however many states it covers.

② Between the rounds: all moves at once

round 0	$N1 \wedge N2$	both away
	∨ **Post:** one robot checks: N → W (flag = 0)	
round 1	$\neg C1 \wedge \neg C2 \wedge \neg(W1 \wedge W2)$	nobody in, not both checked
	∨ **Post:** a checked robot enters (W → C, flag := 1), or the other checks	
round 2	$(\neg C1 \wedge \neg C2) \vee (C1 \wedge N2) \vee (N1 \wedge C2)$	nobody in, or one in, other away
	∨ **Post:** from (W, W, 0): either robot enters	
round 3	$\neg(C1 \wedge C2)$	not both in the cell
	∨ **Post:** the second robot enters too: it checked before the flag was set	
round 4	**TRUE (includes C1 ∧ C2)**	everything: bad state reached ✗

each arrow

Reach := Reach ∨ Post(Reach)

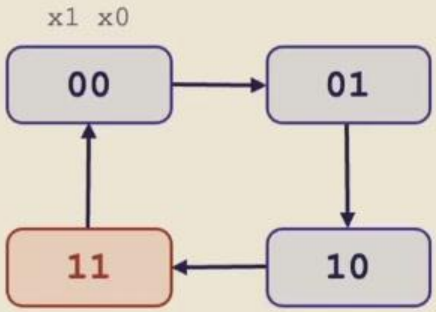
all moves of both robots, applied to the whole group in one calculation

the gold words: what the calculation adds; the tool never lists it

Each arrow is one calculation: all moves, the whole group, at once. Round 3 → 4 is the bug.

② The arrows are a formula too

2-bit counter

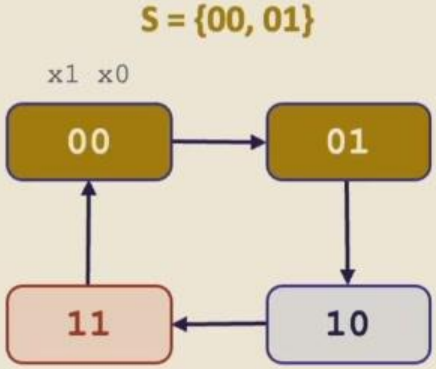


what	as a formula
start state {00}	$\neg x1 \wedge \neg x0$
bad state {11}	$x1 \wedge x0$
the arrows T(x, x′)	$x0' = \neg x0$ and $x1' = x1 \oplus x0$

x′ = value after one step (flip-flop equations; for a PLC: one scan). ⊕ = XOR.
T is TRUE exactly when an arrow goes from x to x′. Property: never 11.

Everything the search needs is a formula: the start states, the bad states, and the arrows T(x, x′).

② One step for a whole group: what we want



Post(S) = every state one arrow away from S

follow the arrows out of S: 00 → 01, 01 → 10
Post({00, 01}) = {01, 10}

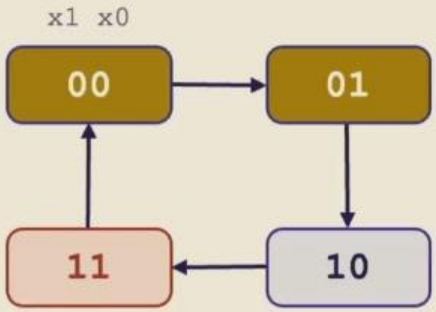
The catch: the tool has no picture, only two formulas: S = ¬x1 and the arrows T. It must find {01, 10} from these alone.

Ask On the picture: what is Post({10, 11})?

Post(S) follows every arrow out of S at once. Next: how to do it with formulas only.

② How the tool does it: AND, try both, rename

S = {00, 01} = ¬x1



step	what we do	result
a AND	S says x1 = 0. Put it into T: x1′ = 0 ⊕ x0 = x0	`x0′ = ¬x0,  x1′ = x0`
b try both	old x0 = 0 → x1′x0′ = 01 old x0 = 1 → x1′x0′ = 10	`x1′ ≠ x0′`
c rename	drop the ′: this is the new group	`x1 ≠ x0  =  {01, 10}` ✓

Same answer as on the picture, found with formulas only. (Tools call step b "forget".)

Post(S) = AND with the arrows, try both old values, rename: one step for a whole group.

② The search loop, with the property to check

Property: never both robots in the cell, G ¬(C1 ∧ C2) → Bad = C1 ∧ C2

1	**Start**	Reach = the start states	`Reach := N1 ∧ N2`
2	**Check**	Bad state among them (AND not empty)? yes → ✗	`Reach ∧ Bad ≠ FALSE ?`
3	**Spread**	Add everything one arrow away (Post)	`Reach := Reach ∨ Post(Reach)`
4	**New?**	Nothing new → ✓ holds. Otherwise back to 2	`Reach unchanged ?`

Like looking for a lost key: search room by room. Key found → done. No new room → the key is not in the house.

Two ways to finish: a bad state is reached (✗ + counterexample), or nothing new appears (✓ holds).

② Running the loop on the robots

Naive protocol (check, then enter)

round	Reach (everything reached so far)	Reach ∧ Bad	result
0	N1 ∧ N2	FALSE	go on
1	¬C1 ∧ ¬C2 ∧ ¬(W1 ∧ W2)	FALSE	go on
2	(¬C1 ∧ ¬C2) ∨ (C1 ∧ N2) ∨ (N1 ∧ C2)	FALSE	go on
3	¬(C1 ∧ C2)	FALSE	go on
4	**TRUE**	**C1 ∧ C2**	✗ violated

Fixed protocol (check + enter at once)

round	Reach	result
0	N1 ∧ N2	go on
1	¬W1 ∧ ¬W2 ∧ ¬(C1 ∧ C2)	go on
2	**same as round 1**	✓ holds

Reach ∧ Bad = FALSE means: no bad state among the states reached so far.

Ask In which step of the loop does each verdict appear?

The verdict comes straight out of the loop: step 2 fires (✗ violated) or step 4 fires (✓ holds).

③ What the search needs from the formula store

job in each round	the question	the store must make it …
1 next states, add them	`Post(Reach), then OR`	small, and fast to calculate with
2 bad state reached?	`is Reach ∧ Bad = FALSE ?`	**easy to see FALSE (the empty group)**
3 nothing new? (stop ✓)	`new Reach = old Reach ?`	**easy to see "same group"**

Checks 2 and 3 ask the same thing: do two formulas describe the same group? (once with FALSE, once with last round's Reach)

> **The store must keep formulas small, calculate fast, and see "same group" at once.**

③ The hard part: is it the same group?

Same group, different texts: `¬(C1 ∧ C2)` and `¬C1 ∨ ¬C2` (and many more)

Comparing the text gives the wrong answer; comparing truth tables needs 2^n rows again.

Ask How would a computer decide quickly that two formulas describe the same group?

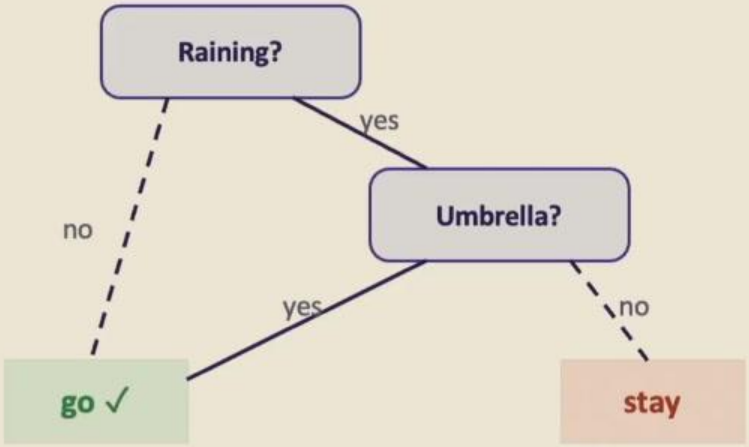
With a BDD (next slides):

- FALSE is always the single 0 box → check 2 "bad state reached?" is instant
- one group has exactly one diagram → check 3 "nothing new?" is "same diagram?"
- small, and AND, OR, forget work directly on it → job 1 is fast

We need a standard form: one small drawing per group, so 'same group' = 'same drawing'. That is the BDD.

③ BDD, the basic idea: a formula as yes/no questions

May I go out? (promise: rain → umbrella)



Binary decision diagram (BDD)

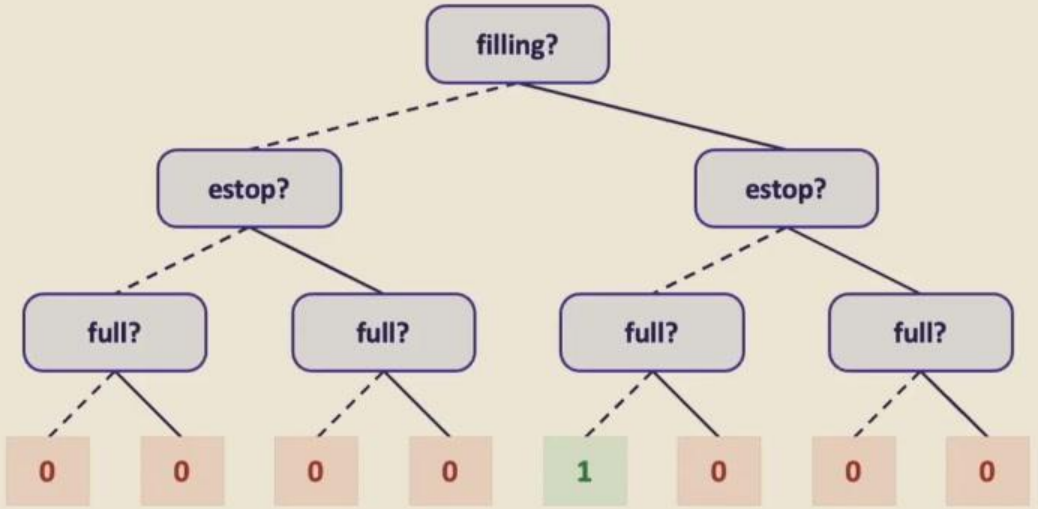
- ask one variable per box
- dashed = no (0), solid = yes (1)
- follow the answers to TRUE / FALSE

Next: build the smallest such diagram for valve = filling ∧ ¬estop ∧ ¬full, step by step.

A BDD is a yes/no flowchart for a formula. The trick is to make it as small as possible.

③ BDD step 1: from the truth table to a decision tree

filling	estop	full	valve
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	**0**	**0**	**1**
1	0	1	0
1	1	0	0
1	1	1	0

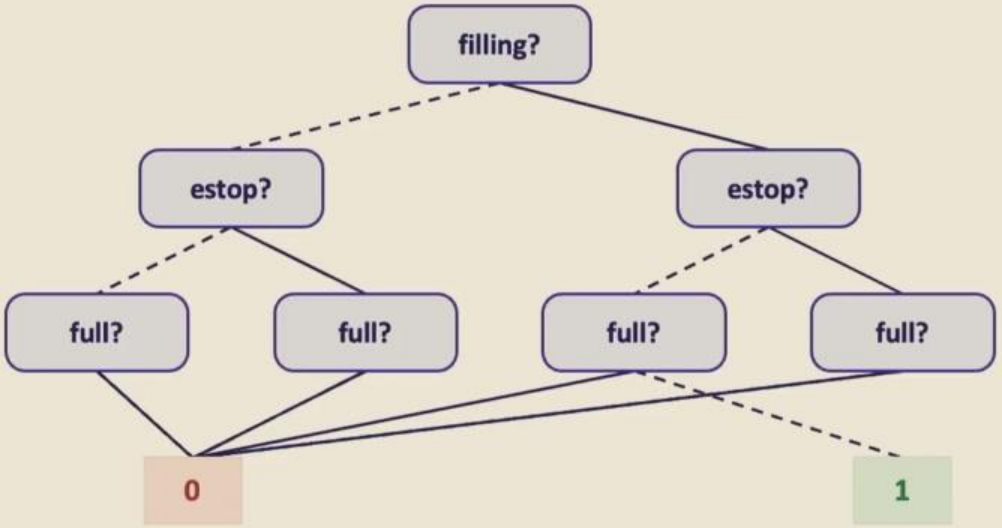


dashed = 0 (no) · solid = 1 (yes)

One question per level: every path is one row of the truth table. 7 questions, 8 answers.

③ BDD step 2: merge equal answers

filling	estop	full	valve
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	**0**	**0**	**1**
1	0	1	0
1	1	0	0
1	1	1	0



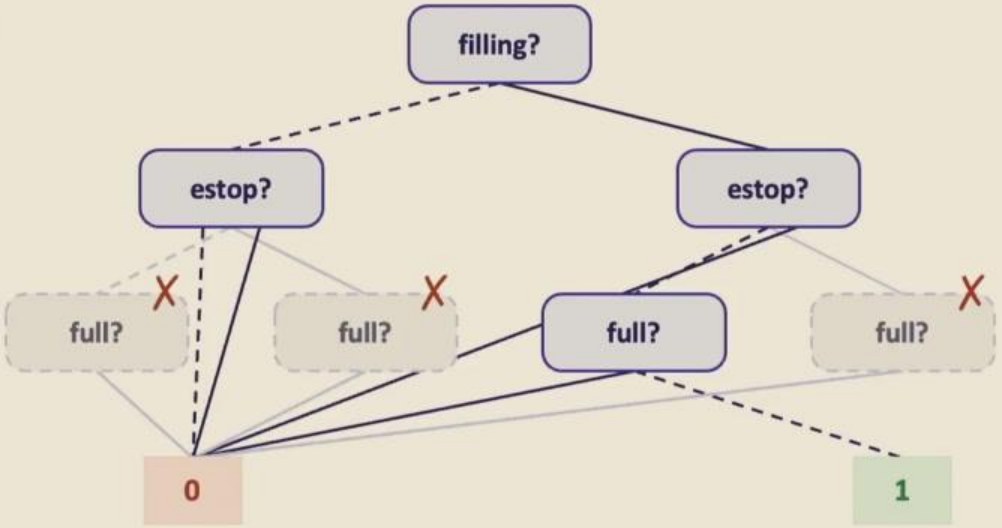
dashed = 0 (no) · solid = 1 (yes)

Ask Look at the full boxes. Which ones are useless now?

③ BDD step 3: delete a useless question

Rule: if both arrows of a question go to the same place, the answer doesn't matter. Delete the question; its incoming arrow goes straight there.

Here: three full? boxes have both arrows to 0 (X).



dashed = 0 (no) · solid = 1 (yes)

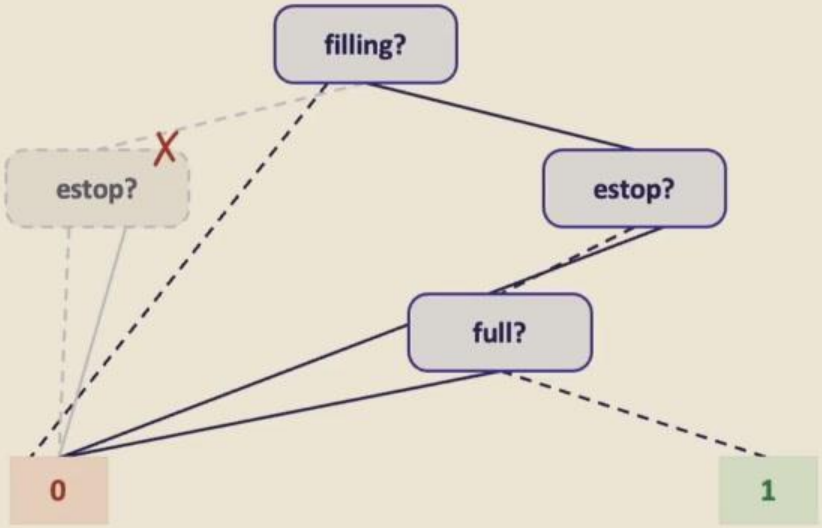
Ask Now look at the left estop? box. What do you notice?

③ BDD step 4: repeat until nothing changes

Same rule again: the left estop? now has both arrows to 0. Delete it.

Nothing else can be deleted: every remaining question has two different targets. Stop.

(Bigger examples also share identical boxes; not needed here.)

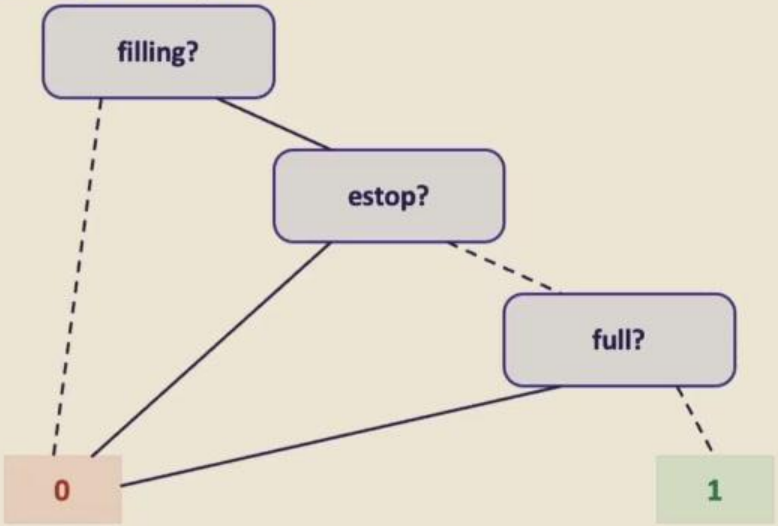


dashed = 0 (no) · solid = 1 (yes)

Delete useless questions, again and again, until none is left: 7 questions shrink to 3.

③ BDD step 5: the result, and why tools use it

valve = filling ∧ ¬estop ∧ ¬full



why tools use BDDs	
small	3 questions instead of 8 rows
unique	same formula → same diagram: "did anything change?" is instant
computable	AND, OR, NOT work directly on diagrams

Bryant 1986. Burch, Clarke, McMillan, Dill & Hwang 1990: 10^{20} states and beyond. NuSMV uses BDDs by default.

Reduced BDD = the smallest yes/no flowchart for a formula, and it is unique for a fixed question order.

③ BDD: the order of the questions matters

"At least one line can run": `(m1 ∧ c1) ∨ (m2 ∧ c2) ∨ (m3 ∧ c3)`

m = motor ready, c = conveyor clear, for lines 1–3

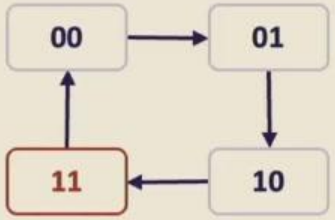
question order	BDD size, 3 lines	8 lines
`m1, c1, m2, c2, m3, c3  (line by line)`	6 questions	16
`m1, m2, m3, c1, c2, c3  (all motors first)`	14 questions	510

Line by line, the diagram only remembers "found a ready line or not". Motors first, it must remember which motors were ready.

Same formula, different question order: 16 vs 510 questions. Good orders keep related variables together.

③ The search with BDDs, step by step: the setup

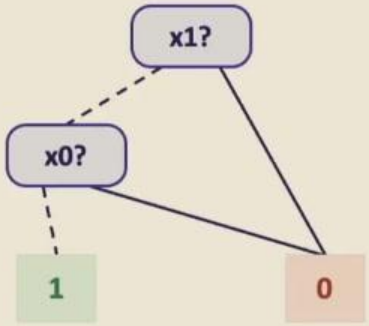
the counter (states x1 x0)



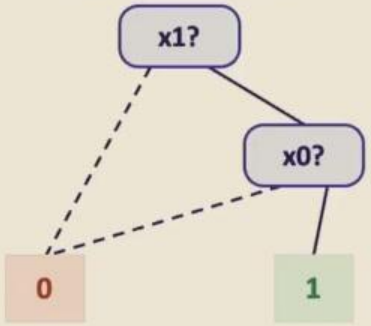
start: 00 **property:** never 11

question order: x1 first, then x0

Start = {00} = ¬x1 ∧ ¬x0



Bad = {11} = x1 ∧ x0



dashed = no (0) · solid = yes (1)

Ask Follow 00 through the Start diagram, and 11 through the Bad diagram. Where do you end?

Start and Bad are small diagrams. From now on the tool works only on diagrams.

③ How to compute Reach ∧ Bad on two diagrams

Like two guards at a gate: you may pass only if both say yes. Ask both the same questions; as soon as one says no, stop.

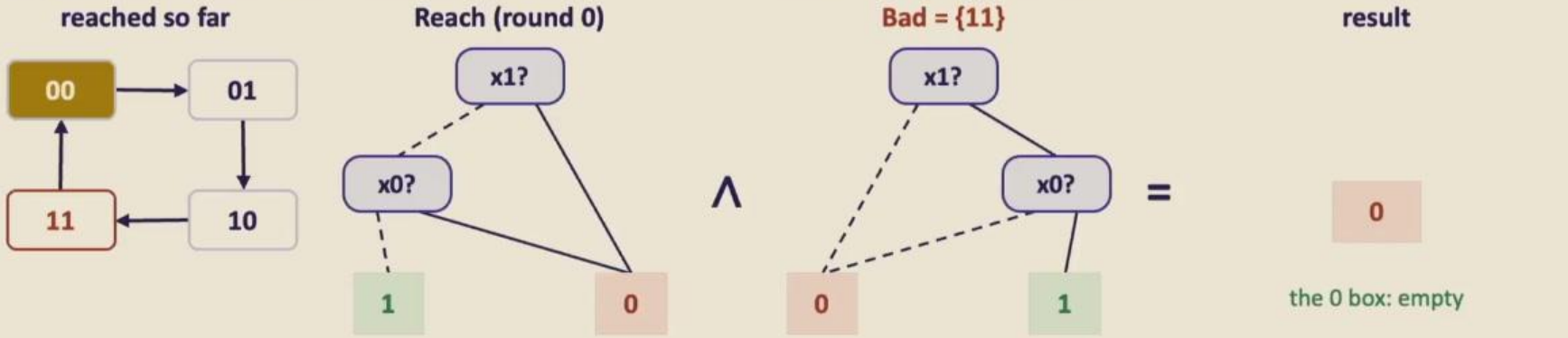
follow the same answers in both diagrams; then …	the result (Reach ∧ Bad)
1 one diagram reaches 0	0: stop, the other does not matter
2 one diagram reaches 1	whatever the other diagram says
3 both still ask a question	ask the next question in both

every path ends in 0 → result = the 0 box
no reached state is bad

some path ends in 1 → not the 0 box
that path names a reached bad state ✗

Reach ∧ Bad = the reached states that are bad. Empty (the 0 box) means: no bad state yet.

③ Round 0: is a bad state already reached?

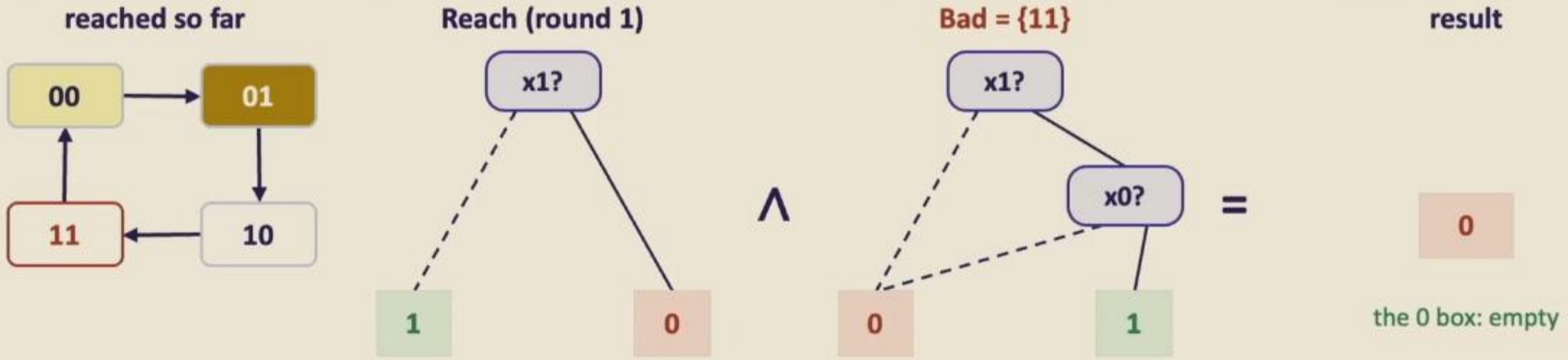


follow the answers	Reach says	Bad says	Reach ∧ Bad
`x1 = no`	asks x0	0	**0 (rule 1: Bad says 0)**
`x1 = yes`	0	asks x0	**0 (rule 1: Reach says 0)**

new? first round → go on; next round adds Post({00}) = {01}

Every path ends in 0: the AND is the 0 box, so no bad state is reached yet.

③ Round 1: add the new states; the diagram shrinks



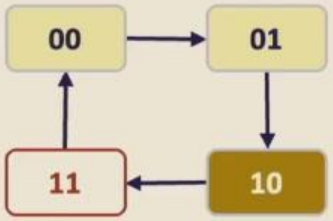
follow the answers	Reach says	Bad says	Reach ∧ Bad
`x1 = no`	1	0	**0 (rule 1: Bad says 0)**
`x1 = yes`	0	asks x0	**0 (rule 1: Reach says 0)**

new? the Reach diagram changed → go on to round 2

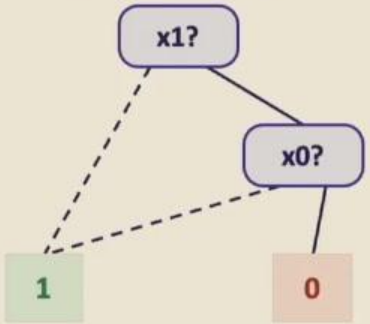
Bigger group, smaller diagram. The AND is still the 0 box: no bad state yet.

③ Round 2: one more state, one more question

reached so far

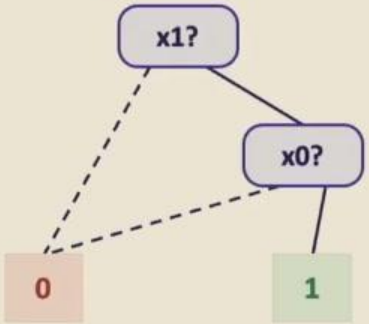


Reach (round 2)



∧

Bad = {11}



=

result

0

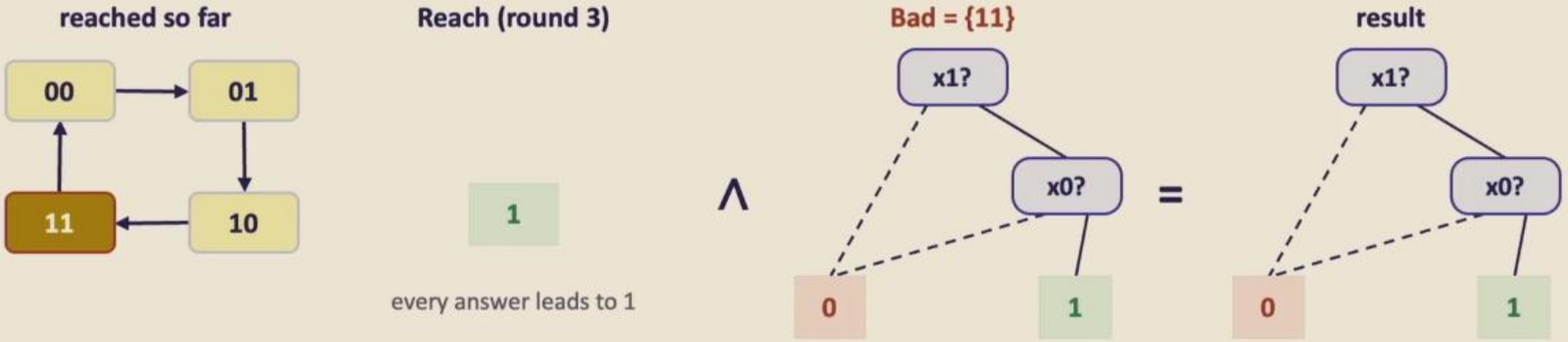
the 0 box: empty

follow the answers	Reach says	Bad says	Reach ∧ Bad
`x1 = no`	1	0	0 (rule 1)
`x1 = yes, x0 = no`	1	0	0 (rule 1)
`x1 = yes, x0 = yes`	0	1	0 (rule 1)

x1 = yes: both diagrams still ask → rule 3: ask x0 in both. new? changed → go on to round 3

Both diagrams still asking → ask the next question in both. Still the 0 box.

③ Round 3: the AND is not the 0 box: violated



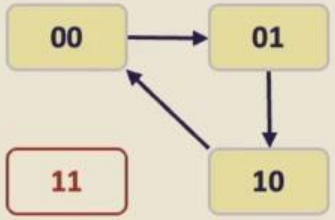
follow the answers	Reach says	Bad says	Reach ∧ Bad
`x1 = no`	1	0	0
`x1 = yes, x0 = no`	1	0	0
`x1 = yes, x0 = yes`	1	1	1 ← bad state 11

Reach is 1 everywhere → rule 2: the result is the Bad diagram. Counterexample: 00 → 01 → 10 → 11

One path ends in 1: not the 0 box, so violated. That path names the bad state 11.

③ The fixed counter: nothing new, so it holds

fixed: 10 → 00



round	Reach	Reach ∧ Bad	new?
0	{00}	0 box	yes
1	{00, 01} = ¬x1	0 box	yes
2	{00, 01, 10}	0 box	yes
3	{00, 01, 10}	0 box	same diagram ✓

bad state? always the 0 box

nothing new → ✓ holds for every run

Two cheap questions on diagrams decide: "is the AND the 0 box?" (✗) and "same diagram?" (✓).

Idea 3, the basic idea: look for one bad run of k steps

	Ideas 1–2 (ripples, groups)	Idea 3 (bounded)
question	which states can ever be reached?	is there a bad run of exactly k steps?
work	the whole reachable set	one run of length k
answer	holds ✓ or bug ✗	bug ✗ with its run, or none of length k

Everyday: "Is there a route from A to F with 3 stops?" You need one route, not the whole map.

Bounded model checking: instead of everything reachable, search for one bad run of a fixed length k.

BMC step 1: one copy of the variables for each step

	step 0	step 1	step 2	step 3	step 4
robot 1	`R1@0`	`R1@1`	`R1@2`	`R1@3`	`R1@4`
robot 2	`R2@0`	`R2@1`	`R2@2`	`R2@3`	`R2@4`
flag	`flag@0`	`flag@1`	`flag@2`	`flag@3`	`flag@4`

R1@2 = where robot 1 is at step 2. 5 time steps × 3 variables = 15 unknowns. Each column is one step, like a PLC timing diagram.

Unrolling: a run of k steps becomes k + 1 copies of the state variables.

BMC step 2: write the rules as one formula

Start

`R1@0 = N ∧ R2@0 = N ∧ flag@0 = 0`
the run begins in the start state

Moves

`T(step 0, step 1) ∧ … ∧ T(step 3, step 4)`
each step: one robot makes one protocol move

Bad

`R1@4 = C ∧ R2@4 = C`
the run ends with both in the cell

`Start ∧ Moves ∧ Bad can be made TRUE? ⇔ a bad run of 4 steps exists`

One formula: TRUE exactly for the runs of k steps that start right, follow the rules and end bad.

BMC step 3: how a SAT solver fills in the blanks

SAT = "can this formula be made TRUE?" (satisfiable). a, b, c stand for cells like R1@2 = W.

$$(a \vee b) \wedge (\neg a \vee c) \wedge \neg c$$

step	reason	value
1	¬c must be TRUE	c = F
2	(¬a ∨ c) with c = F needs ¬a	a = F
3	(a ∨ b) with a = F needs b	b = T
✓	all three parts are TRUE	solution

Like Sudoku

1. write in what is forced
2. if nothing is forced, guess
3. contradiction? undo the guess, try the other value

Modern solvers do this for millions of variables.

A SAT solver propagates what is forced, guesses when stuck, and backtracks on contradictions.

BMC step 4: try it yourself on the robots

	step 0	step 1	step 2	step 3	step 4
robot 1	N	?	?	?	C
robot 2	N	?	?	?	C
flag	0	?	?	?	?

Each column is one step of the run, like a PLC timing diagram.

Like Sudoku

Fixed cells: the start, and both robots in the cell at step 4.
Rule: in each step one robot makes one move of its protocol.
Find the ? cells.

A SAT solver fills in such TRUE/FALSE puzzles, with millions of cells.

Pairs · 3 min Fill in steps 1–3. Is there a solution with only 3 steps?

Bounded model checking: "is there a k-step run that ends in a bad state?" as one puzzle.

BMC step 5: the solver's answer is the counterexample

	step 0	step 1	step 2	step 3	step 4
robot 1	N	**W**	**W**	**C**	**C**
robot 2	N	**N**	**W**	**W**	**C**
flag	0	**0**	**0**	**1**	1

steps k	solver says	meaning
1–3	no solution	no collision within 3 steps
4	**solved**	**collision + its timing diagram** ✗

Biere, Cimatti, Clarke & Zhu 1999. Tools: nuXmv, NuSMV (bmc), CBMC, ESBMC (ESBMC-PLC+ for PLCs).

Solved = a bug with its timing diagram. No solution = no bug within k steps, not yet a proof.

BMC step 6: increase k; what "no solution" means

k	naive protocol	fixed protocol
0, 1, 2, 3	no solution	no solution
4	**solution: collision ✗**	no solution
5, 6, …	(already found)	no solution

In practice: bounded checking finds bugs fast; BDDs or induction prove there are none.

"No solution up to k"

= no bug in the first k steps.
Not a proof for longer runs.

For a proof: an induction argument (k-induction), or knowing that no run is longer than k.

Increase k until a bug appears. "No bug up to k" is evidence, not yet a proof.

Three ideas, three families of tools

family	everyday picture	good at	tools
Explicit-state	ripples: mark states one by one	software, protocols	SPIN, TLC
Symbolic (BDD)	K-map groups: many states per formula	hardware-like systems	SMV, NuSMV
Bounded (SAT)	Sudoku: fill in a k-step timing diagram	finding bugs fast	nuXmv, CBMC, ESBMC

Same promise for all: model + property in; ✓ holds or ✗ counterexample out.

NuSMV, our tool, works with groups (BDDs) and can also solve the puzzle (bounded).

Take-aways: how model checkers work

The goal	search all reachable states for a bad one; report a run if found.
Idea 1: ripples	explicit search, state by state; shortest counterexample; state explosion.
Idea 2: groups	formulas for sets of states, stored as BDDs; one step = AND, try both, rename; stop when nothing new appears.
Idea 3: puzzle	unroll k steps into one formula; a SAT solver finds the bad run; "none up to k" is not a proof.
In this course	NuSMV: BDD-based by default, bounded checking on request.

Same promise for all three: model + property in; ✓ holds or ✗ counterexample out.



浙江大学 · Zhejiang University

PROGRAM VERIFICATION

Proving Programs Correct

Program verification with Dafny, for every input and every scan

Where we are: three ways to trust PLC code

	Testing	Model checking	Program verification
what it covers	a few chosen runs	all runs of a finite model	**all runs, any data size**
who does the work	you pick the tests	the machine, automatically	**you write the specification, the machine proves**
you get	pass / fail	✓ or ✗ + counterexample	**✓ + a proof, or the line that fails**

Today: one tank throughout. ① proof by hand, the solver ② Dafny ③ numbers, loops, AI

> **Program verification covers all runs, like model checking, but with a proof instead of a search.**

What model checking cannot do easily

Big numbers

Tank level from an analog sensor: one INT = 65,536 values. Three INT variables ≈ 280 trillion combinations.

"For every n"

n level sensors on one tank, n tanks in a plant: the logic must work for every n. A model checker checks one n at a time.

"Why does it hold?"

The model checker says ✓ but gives no reason. A proof is an argument you can read.

Ask Where in your PLC code do INT or REAL calculations appear?

For numbers, "any n" and explanations, we need an argument that covers all cases at once: a proof.

Our tank again: FB_Fill and its requirements

the LLM's FB_Fill (opening), one scan

```
IF start THEN filling := TRUE; END_IF;
IF full OR stop THEN filling := FALSE; END_IF;
valve := filling AND NOT estop AND NOT full;
```

variable	meaning
`start stop estop`	input: TRUE = pressed
`full`	input: TRUE = tank full
`filling`	memory: fill request (kept)
`valve`	output: TRUE = open

	requirement	formula
R1	never open while full	`full ⇒ ¬valve`
R2	E-Stop closes the valve	`estop ⇒ ¬valve`
R3	opens only on a new Start press	`(block ②)`
R4	new Start press when safe opens it	`(homework)`
R5	Stop closes the valve	`stop ⇒ ¬valve`

⇒ reads "implies": if … then. ¬ = NOT. estop ⇒ ¬valve: if the E-Stop is pressed, the valve is not open.

Today we prove these requirements about this code: first by hand, then with Dafny.

Testing vs. proving: our tank, requirement R2

the LLM's FB_Fill, one scan

```
IF start THEN filling := TRUE; END_IF;
IF full OR stop THEN filling := FALSE; END_IF;
valve := filling AND NOT estop AND NOT full;
```

R2: the E-Stop closes the valve in the same scan: $\texttt{estop} \Rightarrow \neg\texttt{valve}$

Testing

a test (e.g. T4) tries one situation

Proving

covers every situation at once

Tests try situations one by one. A proof covers all of them: let us do one, step by step.

Proof step 1: what exactly do we want to show?

the LLM's FB_Fill, one scan

```
IF start THEN filling := TRUE; END_IF;                (* statement 1 *)
IF full OR stop THEN filling := FALSE; END_IF;        (* statement 2 *)
valve := filling AND NOT estop AND NOT full;          (* statement 3 *)
```

We may assume

estop = TRUE (E-Stop pressed)
any start, stop, full
any filling (the memory from the last scan)

We must show

valve = FALSE after the scan

Ask How many combinations of start, stop, full and filling are there?


USB-C
IN : NO SIGNAL
OUT : 3840X2160P 60


No Signal



24.9cm X 25.4cm
冊絹設色
台北故宮博物院藏

《苹婆山鸟图》
黄筌 · 宋代

15:50
2026/10/26